surprising and weird if they didn't. It's because we're assuming that `flatMap` obeys an *associative law*:

```
x.flatMap(f).flatMap(g) == x.flatMap(a => f(a).flatMap(g))
```

And this law should hold for all values `x`, `f`, and `g` of the appropriate types—not just for `Gen` but for `Parser`, `Option`, and any other monad.

11.4.2 Proving the associative law for a specific monad

Let's *prove* that this law holds for `Option`. All we have to do is substitute `None` or `Some(v)` for `x` in the preceding equation and expand both sides of it.

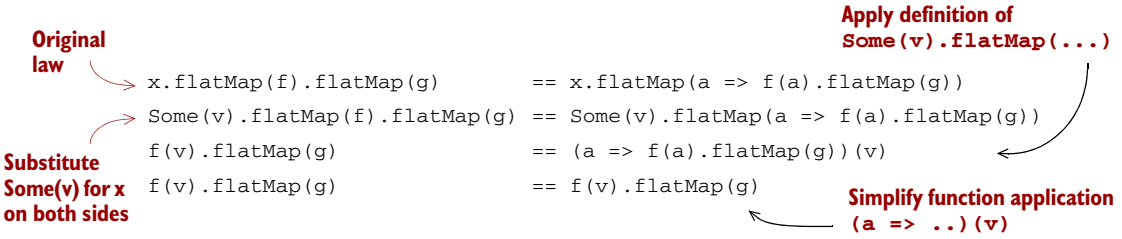
We start with the case where `x` is `None`, and then both sides of the equal sign are `None`:

```
None.flatMap(f).flatMap(g) == None.flatMap(a => f(a).flatMap(g))
```

Since `None.flatMap(f)` is `None` for all `f`, this simplifies to

```
None == None
```

Thus, the law holds if `x` is `None`. What about if `x` is `Some(v)` for an arbitrary choice of `v`? In that case, we have



Thus, the law also holds when `x` is `Some(v)` for any `v`. We're now done, as we've shown that the law holds when `x` is `None` or when `x` is `Some`, and these are the only two possibilities for `Option`.

KLEISLI COMPOSITION: A CLEARER VIEW ON THE ASSOCIATIVE LAW

It's not so easy to see that the law we just discussed is an *associative* law. Remember the associative law for monoids? That was clear:

```
op(op(x,y), z) == op(x, op(y,z))
```

But our associative law for monads doesn't look anything like that! Fortunately, there's a way we can make the law clearer if we consider not the monadic values of types like `F[A]`, but monadic *functions* of types like `A => F[B]`. Functions like that are called *Kleisli arrows*,[7] and they can be composed with one another:

```
def compose[A,B,C](f: A => F[B], g: B => F[C]): A => F[C]
```

[7] *Kleisli arrow* comes from category theory and is named after the Swiss mathematician Heinrich Kleisli.